PyTorch Implementation of a Performer FAVOR+ attention class

This paper documents my implementation in Python using the PyTorch library of a Performer attention framework as defined *Rethinking Attention With Performers – ICLR 2021*. It provides a core engine to be plugged in lightweight open source models such as TinyLlaMA for computation/performance tradeoff analyses. Python code and plugins on various models can be found on the repository : https://github.com/Antoinechss/performer-attention-llm.git

Class initialization, parameters and projections

```
def __init__(self, dim, num_heads, head_dim, num_features):
    super().__init__()

    self.dim = dim
    self.num_heads = num_heads
    self.head_dim = head_dim
    self.num_features = num_features

    self.sample_features_ORF()

    inner_dim = num_heads * head_dim

    self.q_proj = nn.Linear(dim, inner_dim, bias=False)
    self.k_proj = nn.Linear(dim, inner_dim, bias=False)
    self.v_proj = nn.Linear(dim, inner_dim, bias=False)
    self.out_proj = nn.Linear(self.num_heads * self.head_dim, self.dim)
```

- dim = size of token embeddings, differs from a model to another as it depends of the encoder used.
- num heads = number of attention heads running in parallel
- head dim = dimension of each query/key/value vector within one head
- num features = number of features sampled for approximation i.e. the m parameter that can be tuned by the operator to reduce or augment the computational cost.
- inner dim = total dimension of all attention heads combined

Q, K and V projections are learned during training. nn.linear layers create learnable linear transformations W_q, W_k, W_v that each map input vector X into query, key and value vectors.

self.sample_features_ORF() computes the sampling at initialization using ORF FAVOR+ method (see function detailed analysis). Sampling feature vector omega is then stored as a class variable in a buffer.

out_proj : This is the final projection layer in multi-head attention. It takes the concatenated outputs of all attention heads and maps them back to the original embedding dimension. Each head has learned different patterns (syntax, semantics, positions etc. But they are just concatenated, not yet combined. This layer Blends all heads together, learns how much each head should contribute and returns to the standard embedding size so the rest of the model can continue.

Orthogonal Random Features (ORF) FAVOR+ Sampling

```
def sample_features_ORF(self):
        blocks = []
        while len(blocks) * self.head_dim < self.num_features:
            G = torch.randn(self.head_dim, self.head_dim)
            Q, _ = torch.linalg.qr(G)
            blocks.append(Q.T)
        stacked_blocks = torch.cat(blocks, dim=0)
        omega = stacked_blocks[: self.num_features]
        self.register_buffer("omega", omega)
```

This function performs Orthogonal Random Features Sampling. It builds a projection matrix Ω of size (head dimension) x (number of features). We build several orthogonal blocks that are then stacked together.

Looping until enough features are generated

1. Generate a Gaussian matrix of size (head dim) x (head dim). Its entries are $\mathcal{N}(0,1)$
2. Apply QR decomposition to G to retrieve the orthogonal matrix Q (columns are orthonormal)
3. Store its transpose (orthogonal rows) in the blocks

The function then stacks all the blocks together and trim the stacked result to the number of desired features. The register_buffer saves Ω as a class variable, can be reused as self.omega in other functions.

Orthogonal Random Features (ORF) FAVOR+ Sampling

Function phi performs the feature mapping : $\phi(x) = \frac{1}{\sqrt{m}} \exp\left(\Omega x - \frac{\|x\|^2}{2}\right)$

That is then used to form the unbiased estimator of $\exp(x^\top y)$.

$$\phi(x)^\top \phi(y) = \frac{1}{m} \sum_{r=1}^{m} \exp\left(\omega_r^\top x - \|x\|^2/2\right) \exp\left(\omega_r^\top y - \|y\|^2/2\right)$$

```
    def phi(self, x):
        proj_x = torch.einsum("bhnd, md -> bhnm", x, self.omega)
        norm_x = 0.5 * (x**2).sum(dim=-1, keepdim=True)
        phi = torch.exp(proj_x - norm_x) / math.sqrt(self.num_features)
        return phi
```

- Project x onto approximation space : compute $w_i^T * x$ for i in $[1, m]$ for every batch b, head h and token n
- Square every coordinate and sum along d dimension to compute the norm
- keepdim=True conserves dimension even after summing for subtraction against Ωx

Forward Pass

```python
def forward(self, x):
        B, N, _ = x.shape

        q = self.q_proj(x)
        k = self.k_proj(x)
        v = self.v_proj(x)

        q = self.reshape_heads(q)
        k = self.reshape_heads(k)
        v = self.reshape_heads(v)

        phi_q = self.phi(q)
        phi_k = self.phi(k)

        kv = torch.einsum("bhnm,bhnd->bhnmd", phi_k, v)
        kv_cumsum = kv.cumsum(dim=2)
        k_cumsum = phi_k.cumsum(dim=2)

        out = torch.einsum("bhnm,bhnmd->bhnd", phi_q, kv_cumsum)

        z = 1 / (torch.einsum("bhnm,bhnm->bhn", phi_q, k_cumsum) + 1e-6)
        z = z.unsqueeze(-1)
        out = out*z

        out = out.transpose(1, 2).contiguous().view(B, N, -1)

        out = self.out_proj(out)

        return out
```

1. Retrieve B, N, _ = batch size, sequence length, embedding dimension
2. Compute the three linear projections for queries keys and value vectors $Q = XW_q, K = XW_k, V = XW_v$. Of shapes $(B, N, dim = num_heads * head_dim)$
3. Split into individual heads for parallel learning of different patterns with reshape_heads function
4. Apply feature map ϕ. This step reduces dimensions from (B, H, N, D) to (B, H, N, M) with $M << D$ the number of chosen features for sampling.
5. Compute the key value interaction by for each position, combining feature representation of keys with values.
6. **Causal Accumulation** : compute prefix sums on the kv vectors for causal attention mechanism. At step t, the model only sees tokens from past and present.
7. out is the numerator computation $\phi(q_i)^\top \sum_{j=1}^{i} \phi(k_j) v_j$
8. z is the **normalization** term where $z_i = \sum_{j=1}^{i} \phi(k_j)$ Final attention output $= out * z$
9. Merge heads and project back to mix information learned across heads

Reshaping Function

```python
def reshape_heads(self, x):
        B, N, _ = x.shape
        return x.view(B, N, self.num_heads, self.head_dim).transpose(1, 2)
```

This function is reformatting tensors to enable multi-head attention

- input shape $= (B, N, H * D)$, all heads are flattened together
- splits heads with .view() function, shape becomes (B, N, H, D). Separate big vector into H smaller vectors (one per head)
- Reorder dimensions from (B, N, H, D) to (B, H, N, D) with .transpose(1,2)

Model Independant Core Performer Attention Class

```
def __init__(self, head_dim, num_features):
    super().__init__()
    self.head_dim = head_dim
    self.num_features = num_features

    self.sample_features_ORF()

def sample_features_ORF(self):
    blocks = []
    device = torch.device("cpu")
     while len(blocks) * self.head_dim < self.num_features:
        G = torch.randn(self.head_dim, self.head_dim, device=device)
        Q, _ = torch.linalg.qr(G)
        blocks.append(Q.T)
        omega = torch.cat(blocks, dim=0)[:self.num_features]
        self.register_buffer("omega", omega)

def phi(self, x):
    proj_x = torch.einsum("bhnd,md->bhnm", x, self.omega)
    norm_x = 0.5 * (x**2).sum(dim=-1, keepdim=True)
    return torch.exp(proj_x - norm_x) / math.sqrt(self.num_features)

def forward(self, q, k, v):
    phi_q = self.phi(q)
    phi_k = self.phi(k)
    kv = torch.einsum("bhnm,bhnd->bhnmd", phi_k, v)
    kv_cumsum = kv.cumsum(dim=2)
    k_cumsum = phi_k.cumsum(dim=2)
    out = torch.einsum("bhnm,bhnmd->bhnd", phi_q, kv_cumsum)
    denom = torch.einsum("bhnm,bhnm->bhn", phi_q, k_cumsum) + 1e-6
    out = out / denom.unsqueeze(-1)
    return out
```